

Алгоритмы и структуры данных

Лекция 1

Введение и основные понятия

Ключарёв Пётр Георгиевич

pgkl@yandex.ru

twitter.com/klyucharev

Кафедра ИУ8 «Информационная безопасность»

МГТУ им. Н.Э. Баумана

2013 г.

Литература

-  Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ.
-  Дасгупта С., Пападимитриу Х., Вазирани У. Алгоритмы
-  Левитин А. Алгоритмы: введение в разработку и анализ
-  Кнут Д. Искусство программирования. Т. 1–3.
-  Скиена С. Алгоритмы. Руководство по разработке.
-  Седжвик Р. Фундаментальные алгоритмы на C++. Ч. 1–4.
-  Седжвик Р. Фундаментальные алгоритмы на C++. Ч. 5
-  Вирт Н. Алгоритмы и структуры данных.
-  Ахо А., Хопкрофт Дж., Ульман Дж. Структуры данных и алгоритмы.

Парадигмы программирования

Императивное программирование

- Программа является описанием алгоритма.
- Алгоритмические модели: машина Тьюринга, машина Поста, Машина с произвольным доступом к памяти, ...
- Языки программирования: C/C++, Java, C#, Fortran, Паскаль, Ассемблер (любой), и др.

Функциональное программирование

- Программа является описанием функций.
- Теоретические основы: λ -выражения, комбинаторная логика, рекурсивные функции, теория категорий.
- Языки программирования: Haskell, Lisp, Erlang, Scheme, F#. и др.

Парадигмы программирования (продолжение)

Логическое программирование

- Программа является описанием логических взаимосвязей.
- Теоретические основы: математическая логика, автоматическое доказательство теорем.
- Языки программирования: Prolog.

Мультипарадигмальное программирование

- Можно смешивать в одной программе разные парадигмы.
- Часто мультипарадигмальные языки менее развиты по сравнению с однопарадигмальными языками.
- Языки программирования: Python.

Алгоритм

Задача

Задача — это некоторое отношение между множеством входов и множеством выходов. Вход и выход — слова над некоторым алфавитом.

Алгоритм (неформальное определение)

Алгоритм — это корректно определенная вычислительная процедура, представляющая собой конечную последовательность действий, на вход которой подается некоторое слово и результатом выполнения которой является словом.

Пример задачи: задача сортировки

Вход: последовательность из n чисел: $(a_1, a_2, \dots, a_n)$.

Выход: перестановка входной последовательности: $(a_{i_1}, a_{i_2}, \dots, a_{i_n})$ такая, что $a_{i_1} \leq a_{i_2} \leq \dots \leq a_{i_n}$.

Алгоритмическая модель

Машина с произвольным доступом к памяти

Машину с произвольным доступом к памяти (Random Access Machine, RAM, Равновероятная адресная машина) можно рассматривать как компьютер, работающий следующим образом:

- алгоритм состоит из конечного числа команд;
- для исполнения простой операции $\{+, *, -, =, if, \dots\}$ требуется один шаг;
- каждое обращение к памяти занимает один шаг;
- объем памяти неограничен;
- один шаг выполняется за одну единицу времени;
- любой алгоритм может быть представлен в виде слова над некоторым алфавитом. Мы будем считать, что алгоритм записан на некотором языке программирования.

Алгоритм

Алгоритм

Алгоритм — центральное понятие информатики.

Алгоритм не зависит от языка программирования, операционной системы, архитектуры вычислительного устройства.

Алгоритмы применяются для совершенно разных задач в различных областях. Приведем некоторые примеры областей.

- Криптография.
- Расшифровка генома человека.
- Функционирование поисковых систем в Интернет.
- Маршрутизация в сетях.
- Поиск и индексация в базах данных.
- Построение изображений в томографии.
- Рендеринг в компьютерных играх.
- Распознавание образов.
- Оптимальное управление техническими системами.
- Логистика.
- Алгоритмический трейдинг.
- Предсказание погоды.
- И многое другое...

Характеристики алгоритмов

Основные характеристики алгоритмов

- Корректность.
- Сложность.
- Требования к памяти.
- Способность к распараллеливанию.

Корректность

Для любого алгоритма требуется доказать его корректность, то есть способность выдать правильный ответ для любого входа.

Сложность

Сложность (время работы) — это количество шагов. Сложность зависит от длины входа. Пусть $\tau(x)$ — время работы на входе x .

- Сложность в худшем случае:

$$T(n) = \max_{x \in X: ||x||=n} \tau(x)$$

- Сложность в лучшем случае:

$$T_b(n) = \min_{x \in X: ||x||=n} \tau(x)$$

- Сложность в среднем:

$$T_m(n) = \frac{1}{|\{x \in X : ||x|| = n\}|} \sum_{x \in X: ||x||=n} \tau(x)$$

Оценки сложности

n	$\log_2 n$	n	$n \log_2 n$	n^2	2^n	$n!$
10	0.003 мкс	0.01 мкс	0.03 мкс	0.1 мкс	1 мкс	3.63 мс
20	0.004 мкс	0.02 мкс	0.08 мкс	0.4 мкс	1 мкс	77 лет
30	0.005 мкс	0.03 мкс	0.14 мкс	0.9 мкс	1 с	$8 \cdot 10^{15}$ лет
40	0.005 мкс	0.04 мкс	0.21 мкс	1.6 мкс	18.3 мин	
50	0.006 мкс	0.05 мкс	0.28 мкс	0.9 мкс	13 дней	
100	0.007 мкс	0.1 мкс	0.6 мкс	10 мкс	$4 \cdot 10^{13}$ лет	
1 000	0.01 мкс	1 мкс	10 мкс	1 мс		
10 000	0.013 мкс	10 мкс	130 мкс	100 мс		
100 000	0.017 мкс	100 мкс	1.7 мс	10 с		
1 000 000	0.020 мкс	1 мс	20 мс	17 мин		
10 000 000	0.023 мкс	10 мс	230 мс	1 день		
100 000 000	0.027 мкс	100 мс	3 с	116 дней		
1 000 000 000	0.03 мкс	1 с	30 с	32 года		

На компьютере, производительностью 10^9 операций в секунду.

Асимптотические обозначения

Обозначения

Рассмотрим две функции $f, g : \mathbb{N} \rightarrow \mathbb{N}$.

Обозначение $f(n) = O(g(n))$ означает, что существуют такие $c \in \mathbb{R}$ и $n_0 \in \mathbb{N}$, что при любом $n > n_0$ выполняется $f(n) \leq c \cdot g(n)$.

Обозначение $f(n) = \Omega(g(n))$ означает, что $g(n) = O(f(n))$.

Обозначение $f(n) = \Theta(g(n))$ означает, что $f(n) = O(g(n))$ и, при этом, $g(n) = O(f(n))$.

Определения

Назовем функцию $f : \mathbb{N} \rightarrow \mathbb{N}$ полиномиальной, если существует такое $k \in \mathbb{R}$, что $f(n) = O(n^k)$. Назовем функцию $f : \mathbb{N} \rightarrow \mathbb{N}$ экспоненциальной, если существует такое $a \in \mathbb{R}$, что $f(n) = \Omega(a^n)$.

Свойства асимптотических обозначений

Сложение

$$f(n) + g(n) = O(\max(f(n), g(n)))$$

$$f(n) + g(n) = \Omega(\max(f(n), g(n)))$$

$$f(n) + g(n) = \Theta(\max(f(n), g(n)))$$

Умножение

$$f(n) \cdot g(n) = O(f(n) \cdot g(n))$$

$$f(n) \cdot g(n) = \Omega(f(n) \cdot g(n))$$

$$f(n) \cdot g(n) = \Theta(f(n) \cdot g(n))$$

Транзитивность O

Если $f(n) = O(g(n))$ и $g(n) = O(h(n))$, то $f(n) = O(h(n))$.

Примеры

$$7n^2 - 15n + 600 = O(n^2)$$

$$7n^2 - 15n + 600 = \Omega(n^2)$$

$$7n^2 - 15n + 600 = \Theta(n^2)$$

$$2^n + 1000n^{100} - 10 = O(2^n)$$

Задачи

- 1 Верно ли, что $(x + y)^2 = O(x^2 + y^2)$?
- 2 Существует ли что-то между полиномиальными и экспоненциальными функциями?

Выполнение алгоритмов

Результат

На данном входе x алгоритм может:

- остановиться (говорят, что алгоритм *применим* ко входу x);
- не остановиться (говорят, что алгоритм *не применим* ко входу x);

Вопрос

Существуют ли задачи, для которых не существует алгоритмов?

Алгоритмически неразрешимые задачи

Алгоритмически неразрешимые задачи

Задачи для которых не существует алгоритма будем называть *алгоритмически неразрешимыми*.

Самоприменимый алгоритм

Назовем алгоритм *самоприменимым*, если он применим к собственной записи.

Задача о самоприменимости

Определить, самоприменим ли данный алгоритм.

Задача об останове

Даны алгоритм M и слово w . Определить, завершит ли алгоритм M свою работу, если на вход ему подать слово w .

Теорема

Задача о самоприменимости алгоритмически неразрешима.

Доказательство.

Предположим, что существует алгоритм A , решающий эту задачу. То есть, алгоритм A определяет, является ли вход самоприменимым алгоритмом и выводит «да» или «нет». Изменим его таким образом, чтобы на входах, соответствующих самоприменимым алгоритмам, он зацикливался. Назовем получившийся алгоритм A_1 . Является ли A_1 самоприменимым? Если это так, то он зациклится, получив свою запись на вход. Следовательно он не является самоприменимым. Если же он не самоприменим, то, по построению, он остановится, получив на вход собственную запись и, следовательно, является самоприменимым. Это противоречие и доказывает теорему. □

Теорема

Задача об останове алгоритмически неразрешима.

Доказательство.

Предположим, что существует алгоритм A , решающий эту задачу. Если мы на его вход подадим два слова M и M , то он определит применимость алгоритма M к самому себе. Но, по предыдущей теореме такого алгоритма не существует. □

Структуры данных

Данные

Любой алгоритм использует данные. Важно эти данные хранить так, чтобы алгоритмы работали достаточно эффективно. Не существует одной — наилучшей структуры данных. В каких-то случаях более подходящей является одна структура данных, в других случаях — другая.

Основные операции над данными

- добавление элемента;
- поиск;
- удаление элемента.

Характеристики структуры данных

- сложность основных операций;
- требования к памяти.

Упражнения

Построить алгоритмы для решения следующих задач.

Возведение в степень

Вход: числа $a \in \mathbb{R}$, $m \in \mathbb{N}$ в двоичном виде.

Выход: a^m

Проверка числа на простоту

Вход: целое число m в двоичном виде.

Выход: «да», если m — простое, иначе «нет».

Решение системы линейных алгебраических уравнений

Вход: количество уравнений, коэффициенты и свободные члены системы.

Выход: набор значений неизвестных, если решение существует и оно единственно.

Упражнения

Оптимизация маршрута робота

Вход: множество S точек на плоскости.

Выход: самый короткий маршрут посещения всех точек из множества S .